Internal Working of append() in Python List

Before understanding this method

A Python list works like a dynamic array. Internally it has two important ideas:

- **size** - number of elements currently present in the list.
- **capacity** - extra internal space reserved by Python for future elements.

When capacity is full, Python creates a bigger internal array, copies old elements, and then performs the operation.

Meaning

append() adds one element at the end of the list.

```
a = [10, 20, 30]
a.append(40)
print(a)   # [10, 20, 30, 40]
```

Internal steps

- Python checks the current size and capacity of the list.
- If free capacity is available, the new element is placed at the next empty position.
- The size of the list is increased by 1.
- If capacity is full, Python allocates a bigger internal array, copies old elements, then adds the new element.

```
Before append:
size = 3, capacity = 5
[10, 20, 30, _, _]

After a.append(40):
size = 4, capacity = 5
[10, 20, 30, 40, _]
```

When capacity is full

```
Before append:
size = 3, capacity = 3
[10, 20, 30]

Python creates bigger space:
[10, 20, 30, 40, _, _]
```

Time complexity

- Usually append() takes O(1).
- Sometimes resizing happens, so that particular operation takes O(n).
- Average/amortized time complexity is O(1).

Key point

append() is fast because it adds at the end and does not shift existing elements.